

Politecnico di Milano

Anno accademico 2012-2013

Ingegneria del Software – Appello del 27 febbraio 2014

Cognome:	
Nome:	Matricola:

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si scrivano le pre- e post-condizioni del metodo `static boolean confronto(ArrayList<Integer> a, ArrayList<Integer> b)`, nei tre casi seguenti:

1. Il metodo ritorna `true` se gli arrayList `a` e `b` contengono gli stessi elementi, magari in ordine diverso, senza ammettere elementi ripetuti;
2. Il metodo ritorna `true` se gli arrayList `a` e `b` contengono gli stessi elementi, magari in ordine diverso, ammettendo elementi ripetuti;
3. Il metodo ritorna `true` se gli arrayList `a` e `b` contengono gli stessi elementi, magari in ordine diverso, ammettendo sia elementi ripetuti sia il caso in cui il numero di occorrenze di un certo elemento `n` in `a` possa essere diverso dal numero di `n` in `b`.

Esercizio 2

Il `Guardaroba` di una persona contiene abiti indossabili sempre (istanze di `Abito`) e abiti stagionali (`AbitoStagionale`), adatti solo per una o più stagioni:

1. Volendo definire un'opportuna gerarchia di abiti, e supponendo che ogni abito offra un metodo `public boolean indossabile(Stagione s)`, sarebbe possibile definire una gerarchia di ereditarietà che rispetti il principio di sostituibilità di Liskov? Motivare brevemente la risposta.
2. Volendo definire un iteratore `abitiEstivi()` per la classe `Guardaroba` che restituisca i soli abiti adatti alla stagione estiva, quali classi definireste per le diverse tipologie di abito e per l'iteratore e come implementereste i suoi metodi?
3. L'uso di una classe *astratta* semplificherebbe la realizzazione dell'iteratore?

Esercizio 3

Si consideri il seguente metodo Java:

```
public static int test(int x, int y) {
   int z = x;

   while (z >= 0) {
      if (z < y || x==-1) break;
      else x = x /z;
      z--;
   }
   return z;
}
```

1. Si disegni il diagramma del flusso di controllo;
2. Si identifichi, se esiste, un insieme di test (minimo) per coprire tutte le istruzioni e le decisioni (branch) del metodo.
3. Si identifichi, se esiste, un insieme di test (minimo) per coprire tutte le istruzioni e le condizioni del metodo.
4. Si spieghi il comportamento del metodo per `x` = `4` e `y` = `-1`.

Esercizio 4

Si progetti il diagramma delle classi UML per realizzare una semplice applicazione che opera su array di interi. Oltre alle solite operazioni di somma degli elementi, ricerca del valore minimo, medio e massimo e di calcolo della mediana, l'applicazione deve consentire la visualizzazione degli array in modi diversi. Ad esempio, sarebbe possibile usare il modo compatto $(3, 45, -8, 56)$, oppure quello standard $(a[0] = 3, a[1] = 45, a[2] = -8, a[3] = 56)$. Si vorrebbe anche poter aggiungere ulteriori formati di stampa personalizzati senza stravolgere le classi esistenti. Come si potrebbe realizzare la struttura delle classi per essere certi di avere un solo motore di calcolo e per poter variare le opzioni di stampa in modo semplice? Quali *design pattern* utilizzereste?